

CPSC 304 Project Cover Page

Milestone #: 4

Date: Mar 30, 2026

Group Number: 45

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Dali Li	86989274	k3w4m	ltylitiayu@hotmail.com
Esther Zhang	75639872	c5k3q	mhua4268@gmail.com
Jiyang Zhang	75224436	q5v3a	zjiyang73@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

Project Summary

Our final project is NutriPal, a nutrition-focused meal planning and recipe management platform backed by an Oracle relational database. The system allows users to add and update recipes, search recipes using flexible filtering conditions, explore ingredient information through dynamic projection, and run analytical queries related to nutrients, dietitians, and beverage ingredients through a web interface. The project successfully connects a Node.js/Express backend with SQL queries and a dynamic frontend, demonstrating core database concepts such as insertion, update, selection, projection, aggregation, nested aggregation, and division.

Final Schema

The final schema remains the same as the initial design, with no modifications made.

- Nutrient(nutrient_id: int, nutrient_name: varchar[100], unit: varchar[50], dailyMaxIntake: int, essentialFlag: boolean)
PK: nutrient_id
nutrient_name(not null)
- IngredientContainsNutrient(nutrient_id: int, ingredient_id: int, quantity_g: decimal)
PK: nutrient_id, ingredient_id
FK: nutrient_id → Nutrient(nutrient_id), ingredient_id → Ingredient(ingredient_id)
quantity_g(not null)
- Ingredient(ingredient_id: int, name: varchar[100], calories: int, originRegion: varchar[100], allergenFlag: boolean)
PK: ingredient_id
name(not null)
- IngredientSubstitutesIngredient(original_ingredient_id: int, substitute_ingredient_id: int)
PK: original_ingredient_id
CK: substitute_ingredient_id
FK: original_ingredient_id → Ingredient(ingredient_id),
substitute_ingredient_id → Ingredient(ingredient_id)
- Member(member_id: int, name: varchar[100], birth_date: date, email: varchar[150])
PK: member_id;
CK: email
name(not null)
- RecipeIncludesIngredient (recipe_id: int, ingredient_id: int, amount_per_100g: decimal)
PK: recipe_id, ingredient_id; FK: recipe_id → Recipe(recipe_id),
ingredient_id → Ingredient(ingredient_id)
amount_per_100g(not null)

- Beverage (recipe_id: int, servingSizeMl: decimal, caffeineMg: decimal, servedTemperature: varchar[20])
PK: recipe_id; FK: recipe_id→Recipe(recipe_id)
- MealSet (set_name: varchar[20], total_calorie_goal: int, target_audience: varchar[20], price: decimal)
PK: set_name
- Recipe(recipe_id: int, name: varchar[20], cookTime: int, spiceLevel: varchar[20], price: decimal, dietitian_id: int)
PK: recipe_id; FK: dietitian_id→Dietitian(dietitian_id)
name (not null)
- Dietitian(dietitian_id: int, name: varchar[50], credential_level: varchar[30], year_of_experience: int)
PK: dietitian_id;
name(not null)
- NutrientGoal (priority_nutrient: varchar[50], target_health_goal: varchar[100])
PK: priority_nutrient;
- MealType (type_name: varchar[50], description: varchar[255], priority_nutrient: varchar[50])
PK: type_name; FK: priority_nutrient → NutrientGoal (priority_nutrient);
- ChildrenMeal(type_name: varchar[50], recommendedAgeMin: int, recommendedAgeMax: int, textureLevel: varchar[50])
PK: type_name; FK: type_name→ MealType(type_name)
- VeganMeal(type_name: varchar[50], plantProteinPerServing: decimal, requiresVeganCertification: boolean, containsEgg: boolean)
PK: type_name; FK: type_name→ MealType(type_name)

Visual Representation

1. Nutrient

	NUTRIENT_ID	NUTRIENT_NAME	UNIT	DAILYMAXINTAKE	ESSENTIALFLAG
1	1	Protein	g	200	1
2	2	Carbohydrate	g	300	1
3	3	Fat	g	70	1
4	4	Vitamin C	mg	1000	1
5	5	Sodium	mg	2300	0

2. IngredientContainsNutrient

	NUTRIENT_ID	INGREDIENT_ID	QUANTITY_G
1	1	1	31
2	2	4	45
3	3	1	3
4	4	2	89
5	5	5	62

3. Ingredient

	INGREDIENT_ID	NAME	CALORIES	ORIGINREGION	ALLERGENFLAG
1	1	Chicken Breast	165	Canada	0
2	2	Broccoli	55	USA	0
3	3	Almond Milk	40	USA	1
4	4	Brown Rice	216	China	0
5	5	Egg	78	Canada	1
6	6	Tofu	60	Canada	1
7	7	Matcha Powder	320	Japan	0
8	8	Coffee Beans	2	Brazil	0
9	9	Cocoa Powder	228	Ghana	0
10	10	Mixed Berries	50	Canada	0

4. IngredientSubstitutesIngredient

	ORIGINAL_INGREDIENT_ID	SUBSTITUTE_INGREDIENT_ID
1	1	3
2	3	1
3	4	2
4	2	4
5	6	5

5. Member

	MEMBER_ID	NAME	BIRTH_DATE	EMAIL
1	1	Alice Chen	1995-04-12	alice@email.com
2	2	Bob Li	1990-08-23	bob@email.com
3	3	Cathy Wang	1988-11-15	cathy@email.com
4	4	David Zhang	2000-01-05	david@email.com
5	5	Eva Liu	1993-07-19	eva@email.com

6. RecipeIncludesIngredient

	🔗 RECIPE_ID ▼	🔗 INGREDIENT_ID ▼	📄 AMOUNT_PER_100G ▼
1	1	1	150
2	1	4	100
3	2	2	120
4	2	4	100
5	2	6	50
6	3	3	200
7	4	5	80
8	4	2	100
9	5	4	50
10	6	3	180
11	6	7	10
12	7	3	150
13	8	3	150
14	8	8	15
15	9	3	180
16	9	9	20
17	10	3	150
18	10	10	80

7. Beverage

	🔗 RECIPE_ID ▼	📄 SERVINGSIZEML ▼	📄 CAFFEINEMG ▼	📄 SERVEDTEMPERATURE ▼
1	6	350	20	Cold
2	7	250	0	Hot
3	8	300	5	Cold
4	9	200	10	Hot
5	10	400	15	Cold

8. MealSet

	🔗 SET_NAME ▼	📄 TOTAL_CALORIE_GOAL ▼	📄 TARGET_AUDIENCE ▼	📄 PRICE ▼
1	Fitness Set	1800	Adults	30
2	Kids Set	1200	Children	20
3	Vegan Set	1500	Vegan	25
4	Low Carb Set	1300	Adults	23
5	Family Set	2200	Family	40

9. Recipe

	🔗 RECIPE_ID 🗑	📄 NAME 🗑	📄 COOKTIME 🗑	📄 SPICELEVEL 🗑	📄 PRICE 🗑	🔗 DIETITIAN_ID 🗑
1		1 Chicken Bowl		30 Medium		13
2		2 Veggie Rice		25 Low		11
3		3 Almond Shake		10 None		7
4		4 Egg Stir Fry		20 High		11
5		5 Rice Soup		40 Low		10
6		6 Matcha Latte		5 None		6
7		7 Ginger Tea		8 Low		4
8		8 Iced Coffee		3 None		5
9		9 Hot Cocoa		10 None		5
10		10 Berry Smoothie		12 None		7

10. Dietitian

	🔗 DIETITIAN_ID 🗑	📄 NAME 🗑	📄 CREDENTIAL_LEVEL 🗑	📄 YEAR_OF_EXPERIENCE 🗑
1		1 Dr. Green	Senior	15
2		2 Dr. Brown	Intermediate	8
3		3 Dr. White	Junior	3
4		4 Dr. Black	Senior	12
5		5 Dr. Blue	Intermediate	6

11. MealType

	🔗 TYPE_NAME 🗑	📄 DESCRIPTION 🗑	🔗 PRIORITY_NUTRIENT 🗑
1	High Protein	Rich in protein foods	Protein
2	Energy Boost	High carb meals	Carbohydrate
3	Low Fat	Reduced fat content	Fat
4	Immune Care	Vitamin C focused	Vitamin C
5	Balanced Diet	Well balanced nutrition	Sodium
6	Super Growth	High protein for growth	Protein
7	Brain Fuel	Healthy fats for brain development	Fat
8	Tummy Comfort	Gentle on digestion	Sodium
9	Strong Bones	Micronutrients for bone health	Vitamin C
10	Bright Vision	Essential vitamins for eyesight	Vitamin C
11	Garden Fresh	Daily dose of greens	Vitamin C
12	Nature Fuel	Plant-based energy source	Carbohydrate
13	Green Power	High plant protein	Protein
14	Harvest Bowl	Seasonal vegetable mix	Carbohydrate
15	Pure Plant	Clean plant-based fats	Fat

12. NutrientGoal

	🔗 PRIORITY_NUTRIENT 🗑	📄 TARGET_HEALTH_GOAL 🗑
1	Protein	Muscle Growth
2	Carbohydrate	Energy Supply
3	Fat	Hormone Regulation
4	Vitamin C	Immune Support
5	Sodium	Electrolyte Balance

13. ChildrenMeal

	TYPE_NAME	RECOMMENDEDAGEMIN	RECOMMENDEDAGEMAX	TEXTURELEVEL
1	Super Growth	5	11	Medium
2	Brain Fuel	4	10	Soft
3	Tummy Comfort	2	7	Smooth
4	Strong Bones	6	13	Medium
5	Bright Vision	4	9	Soft

14. VeganMeal

	TYPE_NAME	PLANTPROTEINPERSERVING	REQUIRESVEGANCERTIFICATION	CONTAINSEGG
1	Garden Fresh	16	1	0
2	Nature Fuel	24	1	1
3	Green Power	19	1	0
4	Harvest Bowl	21	1	0
5	Pure Plant	14	1	0

A list of all SQL queries

All SQL queries used to satisfy the rubric items are in File: team_45/appService.js

Query 1 Insert:

Functions:

checkDietitianExists(dietitianId)

insertRecipe(recipe_id, name, cookTime, spiceLevel, price, dietitian_id)

Lines:

checkDietitianExists(): 88 - 96

insertRecipe(): 98 - 108

SQL:

1. SELECT COUNT(*) FROM Dietitian WHERE dietitian_id = :dietitian_id
2. INSERT INTO Recipe (recipe_id, name, cookTime, spiceLevel, price, dietitian_id)
VALUES (:recipe_id, :name, :cookTime, :spiceLevel, :price, :dietitian_id)

Main Query for Grading:

INSERT INTO Recipe (recipe_id, name, cookTime, spiceLevel, price, dietitian_id)

VALUES (:recipe_id, :name, :cookTime, :spiceLevel, :price, :dietitian_id)

Query 2 Update:

Functions:

fetchAllRecipes()

updateRecipe(recipe_id, name, cookTime, spiceLevel, price, dietitian_id)

Lines:

fetchAllRecipes(): 111 - 120

updateRecipe(): 122 - 161

SQL:

1. SELECT recipe_id, name, cookTime, spiceLevel, price, dietitian_id

FROM Recipe

ORDER BY recipe_id ASC

2. UPDATE Recipe

SET [dynamically constructed non-primary-key attributes]

WHERE recipe_id = :recipe_id

Main Query for Grading:

UPDATE Recipe

SET [dynamically constructed non-primary-key attributes]

WHERE recipe_id = :recipe_id

Query 3 Delete:

Functions:

deleteRecipe(recipeId)

Lines:

310 - 334

SQL:

DELETE FROM Recipe

WHERE recipe_id = :recipeld

Query 4 Selection:**Functions:**

buildRecipeConditionClause(condition, bindIndex)

searchRecipes(conditions)

Lines:

buildRecipeConditionClause(): 164 - 194

searchRecipes(): 196 - 283

SQL:

1. SELECT recipe_id, name, cookTime, spiceLevel, price, dietitian_id

FROM Recipe

WHERE [dynamically constructed conditions with AND / OR grouping]

Examples of dynamically generated conditions:

- LOWER(name) LIKE LOWER(:val0)

- cookTime <= :val1

- price >= :val2

- spiceLevel = :val3

Main Query for Grading:

SELECT recipe_id, name, cookTime, spiceLevel, price, dietitian_id

FROM Recipe

WHERE [dynamically constructed conditions with AND / OR grouping]

Query 5 Projection:

Functions:

getDynamicIngredients(columns)

Lines:

290 - 307

SQL:

SELECT [selected columns] FROM Ingredient

Query 6 Join**Functions:**

searchIngredientByNutrient (nutrientName)

Lines:

381- 400

SQL:

SELECT I.name

FROM Nutrient N

JOIN IngredientContainsNutrient ICN

ON N.nutrient_id = ICN.nutrient_id

JOIN Ingredient I

ON ICN.ingredient_id = I.ingredient_id

WHERE LOWER(N.nutrient_name) = LOWER(:nutrientName)

Query 7 Aggregation with GROUP BY**Functions:**

countRecipesByDietitian()

Lines:

338 - 352

SQL:

```
SELECT r.dietitian_id, d.name, COUNT(r.recipe_id)
FROM Recipe r, Dietitian d
WHERE r.dietitian_id = d.dietitian_id
GROUP BY r.dietitian_id, d.name
```

Description:

Counts the number of recipes contributed by each dietitian. The query groups recipes by dietitian and returns both their ID and name.

Query 8 Aggregation with HAVING**Functions:**

aggregationNutrient (nutrientName, threshold)

Lines:

402 - 424

SQL:

```
SELECT I.name, SUM(ICN.quantity_g)
FROM Nutrient N
JOIN IngredientContainsNutrient ICN
    ON N.nutrient_id = ICN.nutrient_id
JOIN Ingredient I
    ON ICN.ingredient_id = I.ingredient_id
WHERE LOWER(N.nutrient_name) = LOWER(:nutrientName)
GROUP BY I.name
HAVING SUM(ICN.quantity_g) > :threshold
```

Description:

This query finds ingredients that contain a high amount of a specific nutrient.

Query 9 Nested Aggregation with GROUP BY:

Functions:

getFastestDietitian()

Lines:

356 - 375

SQL:

```
SELECT d.name, AVG(r.cookTime) AS avg_cook_time
FROM Dietitian d, Recipe r
WHERE d.dietitian_id = r.dietitian_id
GROUP BY d.dietitian_id, d.name
HAVING AVG(r.cookTime) <= ALL (
    SELECT AVG(cookTime)
    FROM Recipe
    GROUP BY dietitian_id
)
```

Description:

Finds the dietitian with the lowest average cook time using GROUP BY and a nested aggregation with HAVING.

Query 10 Division

Functions:

findCommonIngredientsInAllBeverages()

Lines:

425 - 449

SQL:

```
SELECT I.name
FROM Ingredient I
```

```
WHERE NOT EXISTS (  
    SELECT B.recipe_id  
    FROM Beverage B  
    WHERE NOT EXISTS (  
        SELECT *  
        FROM RecipeIncludesIngredient RII  
        WHERE RII.recipe_id = B.recipe_id  
        AND RII.ingredient_id = I.ingredient_id  
    )  
)
```

Description:

Find ingredients that appear in every beverage recipe.